

NLTK

- What is NLTK? Overview and Applications
 - Installing NLTK (`pip install nltk`)
 - NLTK and Its Ecosystem
 - NLTK modules and functions
 - NLTK Data and Corpora
 - Overview of Text Processing in NLP
 - Setting up NLTK and Downloading Datasets
-

2. Text Preprocessing with NLTK

2.1 Tokenization

- Word Tokenization (`nltk.word_tokenize()`)
- Sentence Tokenization (`nltk.sent_tokenize()`)
- Regular Expressions for Tokenization
- WordPunctTokenizer and TreebankWordTokenizer

2.2 Text Cleaning

- Lowercasing and Case Normalization
- Removing Punctuation and Special Characters
- Removing Stopwords (e.g., from NLTK's stopwords corpus)
- Stemming and Lemmatization
 - Porter Stemmer (`nltk.PorterStemmer()`)
 - Snowball Stemmer
 - WordNet Lemmatizer (`nltk.WordNetLemmatizer()`)

2.3 Text Representation

- Bag of Words (BoW) Model
 - TF-IDF (Term Frequency-Inverse Document Frequency)
 - N-grams and Bigram/Trigram Models
 - Frequency Distribution of Words (`nltk.FreqDist()`)
-

3. POS Tagging and Named Entity Recognition (NER)

3.1 Part of Speech (POS) Tagging

- Understanding POS Tags (e.g., NN, VB, JJ)
- POS Tagging with NLTK (`nltk.pos_tag()`)
- POS Tagging with Custom Models and Rules
- Treebank POS Tagset and Universal POS Tagset

3.2 Named Entity Recognition (NER)

- Introduction to NER
- NLTK's Named Entity Chunking
 - Recognizing Entities: People, Organizations, Locations, etc.
 - Chunking with Regular Expressions (`nltk.RegexpParser()`)
- Named Entity Recognition using pre-trained models in NLTK

3.3 Chunking

- Introduction to Chunking
 - Chunking using Regular Expressions
 - Understanding Chunks: Noun Phrase (NP) Chunking
 - Building Custom Chunkers with NLTK
-

4. Syntax and Parsing

4.1 Understanding Grammar

- Formal Grammar and Syntax in NLP
- Context-Free Grammar (CFG)
- Regular Expressions for Pattern Matching

4.2 Parsing Techniques

- Recursive Descent Parsing
- Top-Down Parsing and Bottom-Up Parsing
- Parsing with NLTK's `nltk.ChartParser()`

4.3 Building Parsers

- Syntax Tree Construction (`nltk.Tree`)
- Parsing Sentences with a CFG Grammar
- Evaluating Parsers and Parsing Ambiguity

- Transformational Rules and Grammar Modification
-

5. Text Classification and Feature Extraction

5.1 Introduction to Text Classification

- Text Classification Overview and Use Cases
- Training and Evaluating Text Classifiers (e.g., Sentiment Analysis)
- Feature Extraction with NLTK

5.2 Feature Engineering

- Extracting Features for Classification (Unigrams, Bigrams)
- Feature Extraction with `nltk.FreqDist()` and `nltk.bigrams()`
- Extracting Lexical Features: Words, Character-level Features
- Using NLTK's `nltk.classify` module

5.3 Building Text Classifiers

- Implementing a Naive Bayes Classifier in NLTK
 - Using Decision Trees and Maximum Entropy Classifiers
 - Model Evaluation: Accuracy, Precision, Recall, F1-Score
 - Handling Imbalanced Datasets
-

6. Text Similarity and Information Retrieval

6.1 Text Similarity

- Cosine Similarity and Euclidean Distance
- Implementing Text Similarity with NLTK
- Using NLTK's `nltk.edit_distance()`

6.2 Information Retrieval with NLTK

- Building a Simple Search Engine
- Indexing Documents and Query Matching
- Implementing Boolean Retrieval Models
- Ranking Documents using TF-IDF and Cosine Similarity

7. Sentiment Analysis

7.1 Introduction to Sentiment Analysis

- What is Sentiment Analysis?
- Classifying Sentiment: Positive, Negative, Neutral
- Common Algorithms and Approaches for Sentiment Analysis

7.2 Sentiment Analysis with NLTK

- Using NLTK's Sentiment Analysis Tools
 - VADER (Valence Aware Dictionary and sEntiment Reasoner) for Sentiment Classification
 - Polarity Scores and Sentiment Scores in NLTK
 - Training a Sentiment Classifier using Naive Bayes in NLTK
-

8. Topic Modeling

8.1 Introduction to Topic Modeling

- What is Topic Modeling?
- Common Topic Modeling Techniques: LDA (Latent Dirichlet Allocation), LSA (Latent Semantic Analysis)

8.2 Topic Modeling with NLTK

- Preprocessing Text for Topic Modeling
 - Using Gensim with NLTK for LDA Topic Modeling
 - Visualizing Topics using pyLDAvis
-

9. Machine Translation

9.1 Introduction to Machine Translation

- Statistical Machine Translation (SMT) Overview
- Rule-based and Data-driven Approaches to Translation

9.2 Implementing a Simple Machine Translation System

- Implementing Word Alignments for Translation
 - Building a Basic Translation System with NLTK
-

10. Text Generation and Language Modeling

10.1 Introduction to Language Modeling

- Language Models: Markov Models and N-grams
- Training a Basic N-gram Language Model with NLTK

10.2 Text Generation with NLTK

- Generating Text with NLTK's N-gram Models
 - Using Markov Chains for Text Generation
 - Creativity in Text Generation (Poetry, Random Text Generation)
-

11. Advanced Topics

11.1 Wordnet and Lexical Semantics

- Introduction to WordNet
- Navigating WordNet with NLTK
 - Synonyms, Antonyms, Hypernyms, Hyponyms
 - WordNet's Lemmas and Synsets
- Semantic Similarity and WordNet-based Measures

11.2 Using NLTK with Deep Learning

- Integration of NLTK with Deep Learning Libraries (TensorFlow, Keras, PyTorch)
- Applying Pre-trained Models for NLP Tasks (Text Classification, Named Entity Recognition)
- Fine-tuning Pre-trained Models (BERT, GPT, RoBERTa) with NLTK Preprocessing

11.3 Text Summarization

- Extractive vs. Abstractive Summarization
- Implementing Extractive Summarization with NLTK

- Using LexRank and TextRank for Extractive Summarization
-

12. NLP Projects and Case Studies

Beginner Projects

- Build a Simple Sentiment Analysis Model
- Implement a Text Preprocessing Pipeline
- Create a Word Frequency Distribution from Text

Intermediate Projects

- Build a Document Classifier for News Articles
- Create a Text Similarity Model for Document Comparison
- Implement Named Entity Recognition with NLTK

Advanced Projects

- Develop a Question Answering System Using NLTK
- Build a Text Generation Model Using N-grams and Markov Chains
- Implement a Machine Translation System from Scratch